

TaskFlow - Session 2 Project Guide

A Simple Guide to the JavaScript We Added

TaskFlow already contained the work from Session 1 and the Session 1 homework.

During Session 2, we added JavaScript so the page could use data and react to the user.

Keep this guide. You can read it at home to remember what we did.

Everything here comes from the real project files. Nothing is invented.

1. Where Did We Start Today?

We did not start from an empty page today.

Before we wrote any JavaScript, TaskFlow already had:

From Session 1

- A header with the title "TaskFlow".
- Three statistic cards: Total Tasks, Completed, Pending.
- Three filter buttons: All, Completed, Pending.
- A task list.
- Green and orange badges.
- A layout that changes on small screens.

From Homework 1

- A footer at the bottom of the page.
- A hover effect on the task rows.
- Extra styles for very small screens.
- Some extra card styles.

So the page already looked finished.

But it could not **do** anything.

The numbers were typed by hand. The tasks were typed by hand. The buttons did nothing when you clicked them.

Today we changed that.

2. What Did Homework 1 Add?

This is a short reminder. These are HTML and CSS ideas you already know.

CORE

1. A footer

In `index.html`, after `</main>`:

```
<footer class="footer">
  <p>TaskFlow – built during the frontend training.</p>
</footer>
```

In `css/style.css`:

```
.footer {
  border-top: 1px solid #e2e5ea;
  color: #6b7280;
  font-size: 14px;
  padding: 20px;
  text-align: center;
}
```

What changed: a small grey line of text at the bottom of the page, with a thin line above it.

Why it is useful: `border-top` is the mirror of the `border-bottom` we used on the header. You reused an idea you already knew.

2. Task rows react to the mouse

```
.task-item:hover {
  border-color: #3b6ef5;
  cursor: pointer;
}
```

What changed: when the mouse is over a task row, the border turns blue and the mouse pointer becomes a hand.

Why it is useful: it tells the user "this row is a thing, not just text".

STRETCH

A second media query for very small screens:

```
@media (max-width: 480px) {  
  /* Each filter button takes a full row so it is easy to tap. */  
  .filter-button {  
    width: 100%;  
  }  
}
```

What changed: on a very narrow phone, each filter button takes a whole row. Big buttons are easier to tap with a finger.

Why it is useful: we already had a media query for 600px. This shows that you can have more than one, for different screen sizes.

A small note: inside this media query there is also a rule for `.task-item` that is **commented out** in our current project. Commented-out code does nothing. If you want to try it, you can remove the `/*` and `*/` around it and see what happens.

CHALLENGE

Two extra rules for the statistic cards:

```
.stats {  
  flex-wrap: wrap;  
}  
  
.stat-card {  
  min-width: 140px;  
}
```

What they mean:

- `flex-wrap: wrap` lets the cards move to a second line when the row gets too narrow.
- `min-width: 140px` stops a card from becoming too thin to read.

Important, so you are not confused: these two rules are in our CSS, but our page still has **three** cards, not four. With only three cards there is enough space, so you do not see any change right now.

The rules are not wrong. They are simply waiting for a fourth card. If you ever add one, the layout will already handle it.

3. Session 2: Why JavaScript?

Here is the whole idea of the three languages:

```
HTML      = What is on the page
CSS       = How it looks
JavaScript = What happens
```

After Session 1 and the homework, our page looked good. But it was **static**. Static means it never changes by itself.

- The number 6 was typed by hand in the HTML.
- The six tasks were typed by hand in the HTML.
- The buttons did nothing.

If we wanted a seventh task, we had to open `index.html` and write more HTML.

Today JavaScript took over those jobs:

- JavaScript now **stores** the tasks.
- JavaScript now **counts** them.
- JavaScript now **builds** the task rows.
- JavaScript now **reacts** when you click a filter button.

4. Variables: `const` and `let`

A variable is a name that stores a value.

We use two words to create variables.

`const`

```
const taskList = document.querySelector("#taskList");
```

`const` means: this name will always point to the same thing.

Why `const` here? We find the task list once when the page opens. We never need to make `taskList` point at a different element later. So `const` is the right choice.

We used `const` for all seven elements we look up, and for the `tasks` list.

`let`

```
let currentFilter = "all";
```

`let` means: this value **will** change later.

Why `let` here? `currentFilter` starts as `"all"`. When the user clicks the **Completed** button, we change it to `"completed"`. A `const` cannot be given a new value, so `let` is the right choice.

We also used `let` inside functions for things that change:

```
let completed = 0;
let pending = 0;
```

These counters go up while we count, so they must be `let`.

A simple rule

Start with `const`. Change it to `let` only when you really need to give the variable a new value.

One extra detail

`const tasks = [ ... ]` uses `const`, but that does **not** mean the list can never change.

`const` only stops us from doing this:

```
tasks = somethingElse; // not allowed with const
```

Adding an item **inside** the list is still allowed. We do exactly that in `getVisibleTasks()` with `visibleTasks.push(task)`.

So: `const` protects the **name**, not the contents.

5. Arrays

An array is a list of values.

We write an array with square brackets `[ ]` and put a comma between the items.

```
const colors = ["red", "green", "blue"];
```

In TaskFlow we store all our tasks in one array:

```
const tasks = [  
  {  
    id: 1,  
    title: "Design the TaskFlow page",  
    completed: true,  
  },  
  {  
    id: 2,  
    title: "Write the HTML structure",  
    completed: true,  
  },  
  // ... four more tasks  
];
```

Why an array?

Imagine we did it without an array:

```
const task1Title = "Design the TaskFlow page";  
const task1Completed = true;  
const task2Title = "Write the HTML structure";  
const task2Completed = true;
```

With six tasks that is twelve variables. And to count the completed ones you would have to write every name by hand.

With an array:

```
const tasks = [ ... ];
```

The tasks belong together, so we keep them in one list.

Then we can use **one loop** to work with every task, no matter how many there are.

`tasks.length` tells us how many items are in the list. We use it for the Total number.

6. Objects

One task needs more than one piece of information. It has a title, and it is finished or not.

An object keeps those pieces together. We write it with curly brackets `{ }`.

```
{  
  id: 1,  
  title: "Design the TaskFlow page",  
  completed: true,  
}
```

Each line inside is a **property**.

A property has two parts:

- the **property name** (`title`)
- the **property value** (`"Design the TaskFlow page"`)

To read a property we use a dot:

```
task.title      // "Design the TaskFlow page"  
task.completed  // true  
task.id         // 1
```

Why an object?

Because the three pieces belong to the **same** task. If we kept them in separate variables, nothing would connect them. With an object, one task is one thing.

So our data has two levels:

```
tasks           -> an array (the list)  
tasks[0]        -> an object (one task)  
tasks[0].title  -> a value  (the text)
```

7. Booleans

A boolean has only two values: `true` or `false`.

We use one in every task:

```
completed: true;    // this task is completed
completed: false;   // this task is still pending
```

Notice there are **no quotes** around `true`.

- `true` is a boolean.
- `"true"` is a piece of text.

They are not the same. This is a very common beginner mistake.

Booleans are useful because we can ask about them directly:

```
if (task.completed) {
```

That reads almost like English: *if the task is completed*.

We can also ask the opposite with `!`, which means "not":

```
!task.completed    // true when the task is NOT completed
```

8. Selecting HTML Elements

Before JavaScript can change something on the page, it has to **find** it.

```
const taskList = document.querySelector("#taskList");
```

Let us break that line into pieces.

document The current webpage. Everything you see lives inside it.

querySelector(...) Find one element on the page.

"#taskList" Which element to find. This is a CSS selector.

means an **id**. So `"#taskList"` means: the element with `id="taskList"`.

If we wanted a class instead, we would use a dot:

```
document.querySelector(".filter-button");
```

. means a **class**.

This is exactly the same selector language you already learned in CSS. You are not learning a new system — you are reusing the one you know.

Here are all seven elements we find in TaskFlow:

```
const taskList = document.querySelector("#taskList");

const totalCount = document.querySelector("#totalCount");
const completedCount = document.querySelector("#completedCount");
const pendingCount = document.querySelector("#pendingCount");

const filterAllButton = document.querySelector("#filterAll");
const filterCompletedButton = document.querySelector("#filterCompleted");
const filterPendingButton = document.querySelector("#filterPending");
```

We do this once, at the top of the file. Then we can use these names anywhere below.

9. Why `querySelector()` and Not `getElementById()` ?

There is another way to find an element by id:

```
document.getElementById("totalCount");
```

and the way we used:

```
document.querySelector("#totalCount");
```

Both work. Both find the same element.

So why did we choose `querySelector()` ?

1. It uses the selector syntax you already know. You learned `#id` and `.class` in CSS. `querySelector()` uses the same thing.

2. It works for ids and classes and more. `getElementById()` can only do ids. With `querySelector()` you use one function for everything:

```
document.querySelector("#totalCount");    // an id
document.querySelector(".filter-button"); // a class
```

3. One consistent style. Our whole file looks the same. You do not have to switch between two different functions depending on what you are looking for.

To be fair: `getElementById()` is **not wrong**. It is a normal, valid way to find an element by id, and you will see it in real projects. We chose `querySelector()` mainly for consistency with the

CSS you already know.

10. Functions

A function is a group of code that does one job.

We wrote four functions today:

Function	Its job
<code>updateStats()</code>	Count the tasks and write the three numbers
<code>getVisibleTasks()</code>	Decide which tasks should be shown
<code>renderTasks()</code>	Build the task rows and put them on the page
<code>setFilter()</code>	Remember the new filter, move the blue colour, redraw

We write a function like this:

```
function updateStats() {  
  // the work goes here  
}
```

Writing a function does **nothing** by itself. It only describes the work.

To make it happen, we have to **call** it:

```
updateStats();
```

This is the most surprising thing for beginners. Defining is not the same as calling.

Why use functions?

- **Easier to read.** The name tells you what the code does.
 - **Easier to reuse.** We call `renderTasks()` from four different places.
 - **Easier to fix.** If the numbers are wrong, you look inside `updateStats()` and nowhere else.
 - **One clear job each.** Each function has one thing to do.
-

11. Function Parameters

A parameter is a value that a function receives.

Look at `setFilter` :

```
function setFilter(newFilter, clickedButton) {  
  currentFilter = newFilter;  
  ...  
  clickedButton.classList.add("active");  
  ...  
}
```

`newFilter` and `clickedButton` are parameters. They are empty names until we call the function.

When we call it, we give it real values:

```
setFilter("completed", filterCompletedButton);
```

Now, inside the function:

- `newFilter` is "completed"
- `clickedButton` is the Completed button

Why is this useful?

Without parameters we would need three almost identical functions: one for All, one for Completed, one for Pending.

With parameters we write the steps **once**, and simply tell the function which filter and which button to use.

12. Loops

A loop runs the same code for every item in a list.

We used this loop:

```
for (const task of tasks) {  
  // do something with this one task  
}
```

Read it as: *for each task in tasks, do this.*

Step by step:

1. Take the first task. Run the code.

2. Take the second task. Run the code.
3. Keep going.
4. Stop when there are no tasks left.

```
Task 1 -> same code
Task 2 -> same code
Task 3 -> same code
Task 4 -> same code
Task 5 -> same code
Task 6 -> same code
```

Inside the loop, `task` is the item we are looking at **right now**.

Why a loop?

Without a loop we would have to write the same code six times, once per task. And if we added a seventh task, we would have to write it again.

The loop does not care how many tasks there are. Six or six hundred, the code is the same.

A note about other ways to loop: `forEach()` is another valid way to go through an array, and you will meet it later. We used `for...of` because it is very direct and easy to read when you are starting out.

13. Conditions

An `if` checks something. An `else` says what to do when the check is false.

```
if (task.completed) {
  completed++;
} else {
  pending++;
}
```

In simple words:

- **If** this task is completed, add one to the completed counter.
- **Otherwise**, add one to the pending counter.

We use the same idea when we build a row:

```
let statusClass = "pending";
let statusText = "Pending";

if (task.completed) {
  statusClass = "completed";
  statusText = "Completed";
}
```

Notice the trick here. We **first assume** the task is pending. Then, only if it is completed, we change the two values.

This is shorter than writing the whole row twice, once for each case.

14. Calculating the Statistics

Here is the real function:

```
function updateStats() {
  let completed = 0;
  let pending = 0;

  for (const task of tasks) {
    if (task.completed) {
      completed++;
    } else {
      pending++;
    }
  }

  totalCount.textContent = tasks.length;
  completedCount.textContent = completed;
  pendingCount.textContent = pending;
}
```

Step by step:

1. Start both counters at zero.

```
let completed = 0;
let pending = 0;
```

They use `let` because they will change.

2. Look at every task and count it.

`completed++` means "add one". It is a short way of writing `completed = completed + 1`.

3. Write the numbers onto the page.

```
totalCount.textContent = tasks.length;
```

We do not count the total by hand. `tasks.length` already knows how many items are in the array.

What you see: the three cards show 6, 2 and 4.

Why this matters: these numbers are now **calculated**. Add a seventh task to the array, save, and all three numbers update by themselves. Nobody has to edit the HTML.

What about the Completion Rate card?

Our page has **three** cards: Total Tasks, Completed and Pending.

There is no Completion Rate card in our HTML, so there is nothing for JavaScript to calculate for it. If you add that card one day, you could calculate it here too.

15. Showing Values on the Page

We used two different ways to put things on the page.

`textContent`

```
totalCount.textContent = tasks.length;
```

`textContent` changes the **text** inside an element.

We use it for the three numbers, because a number is just text. There are no tags in it.

`innerHTML`

```
taskList.innerHTML = html;
```

`innerHTML` replaces everything inside an element, and the browser reads it **as HTML**.

We use it for the task list, because our string really does contain HTML tags: `<li>` , `<span>` , and class names.

The difference in one sentence

- `textContent` -> "this is plain text, show it exactly as it is"
 - `innerHTML` -> "this text contains HTML tags, please build real elements from it"
-

16. Why Both `textContent` and `innerHTML` ?

This was a real decision, so here is the reason.

For a number like Total, `textContent` is enough.

The value is `6` . There are no tags. Using `innerHTML` here would work, but it would ask the browser to look for HTML that is not there. `textContent` says exactly what we mean.

For the task rows, we need `innerHTML` .

Our string contains real tags:

```
<li class="task-item">
  <span class="task-title">...</span>
</li>
```

If we used `textContent` here, the page would show the tags as visible text, like `<li class="task-item">` , instead of building the rows. Try it once — it is a useful thing to see.

A fair note: we should not reach for `innerHTML` every time. When we only need to change text, `textContent` is simpler and clearer. Use the one that matches what you actually have.

(There is one more reason professionals are careful with `innerHTML` : if text ever comes from a stranger, treating it as HTML can be unsafe. Our task titles are written by us, so this is fine here. You will learn more about it later.)

17. Rendering the Tasks

`renderTasks()` is the function that puts the tasks on the page.

Let us build it piece by piece.

Step 1. Ask which tasks should be visible.

```
const visibleTasks = getVisibleTasks();
```

We do not decide here. We ask another function. `renderTasks()` only has to draw.

Step 2. Start with an empty string.

```
let html = "";
```

This is `let` because we will add to it many times.

Step 3. Go through the visible tasks one by one.

```
for (const task of visibleTasks) {
```

Step 4. Decide how this task looks.

```
let statusClass = "pending";
let statusText = "Pending";

if (task.completed) {
  statusClass = "completed";
  statusText = "Completed";
}
```

`statusClass` decides the badge colour (our CSS already has `.completed` and `.pending`).
`statusText` decides the word inside the badge.

Step 5. Add one row to the string.

```
html += `
  <li class="task-item">
    <span class="task-title">${task.title}</span>
    <span class="task-status ${statusClass}">${statusText}</span>
  </li>
`;
```

`+=` means "add this to what we already have".

Step 6. Put the finished string on the page.

```
taskList.innerHTML = html;
```


We do this **once**, at the end, after the loop has finished.

What you see: the six task rows appear, with green and orange badges — exactly like the rows we used to type by hand.

A note about an empty list: in our current project, if a filter matches no tasks, the list area is simply empty. There is no message. That is something you could add later.

18. Template Literals

Look at the back-tick character in the code above: ```

A string written with back-ticks is called a **template literal**.

It can do two things normal quotes cannot.

1. It can hold a value inside `${ }`.

```
`${task.title}`
```

means: *put the value of `task.title` here.*

2. It can go over many lines.

That is why we can write the whole `<li>` block, nicely indented, inside one string.

Why is this easier?

Without template literals we would have to join pieces with `+`:

```
html += "<li class='task-item'><span class='task-title'>" + task.title + "  
</span></li>";
```

That is hard to read, and it is very easy to lose a quote or a `+`.

With a template literal, the HTML looks like HTML. You can see the shape of it.

Careful: the back-tick is not the normal quote. It is usually the key at the top-left of the keyboard, near `Esc`. If you use `"` by mistake, the page will show `${task.title}` as visible text.

19. Events

An event is something that happens on the page.

Examples:

- the user **clicks** a button
- the user **types** in a box
- the page finishes loading

In Session 2 we only used one event: **click**.

Our page has three buttons, and we want something to happen when each one is clicked.

20. `addEventListener()`

`addEventListener()` tells JavaScript:

"When this happens, run this code."

Here is the real code:

```
filterCompletedButton.addEventListener("click", function () {  
    setFilter("completed", filterCompletedButton);  
});
```

Let us break it down.

`filterCompletedButton` The element we are listening to. The Completed button.

`"click"` The event we are waiting for.

`function () { ... }` The code to run when it happens.

In plain English: *when the Completed button is clicked, call `setFilter` with `"completed"` and this button.*

Important: the function inside is **not** run when the page opens. JavaScript keeps it and waits. It only runs when someone actually clicks.

We do the same for all three buttons:

```
filterAllButton.addEventListener("click", function () {  
    setFilter("all", filterAllButton);  
});  
  
filterCompletedButton.addEventListener("click", function () {  
    setFilter("completed", filterCompletedButton);  
});
```

```
filterPendingButton.addEventListener("click", function () {
  setFilter("pending", filterPendingButton);
});
```

Three listeners, but they all call the **same** function with different values.

21. Why `addEventListener()` and Not `onclick` in the HTML?

There is another way to react to a click. You can put it directly in the HTML:

```
<button onclick="showCompleted()">Completed</button>
```

We chose a different way. Our HTML only gives the button an id:

```
<button class="filter-button" id="filterCompleted">Completed</button>
```

and our JavaScript does the rest:

```
filterCompletedButton.addEventListener("click", function () {
  setFilter("completed", filterCompletedButton);
});
```

Why we prefer this:

- 1. Each file keeps one job.** HTML describes what is on the page. JavaScript describes what happens. We do not mix JavaScript code inside HTML tags.
- 2. All the behaviour is in one place.** If you want to know what the buttons do, you open `js/app.js`. You do not have to search through the HTML.
- 3. It is easier to change.** Rename a function, and you change it in the JavaScript file only.
- 4. You can add more than one listener.** With `addEventListener()` you can react to the same click in two different ways if you ever need to. `onclick` holds only one.
- 5. You will see this pattern everywhere.** Larger projects almost always separate HTML and behaviour like this.

To be fair: `onclick` **works**. It is not wrong, and for a tiny example it can be quick and simple. We chose `addEventListener()` because it keeps our HTML and our JavaScript more separate, which helps as a project grows.

22. Filtering the Tasks

We have three filters: **all**, **completed** and **pending**.

Remembering the choice

```
let currentFilter = "all";
```

This variable remembers which filter is selected right now.

It is `let` because it changes every time the user clicks a different button.

Notice the value is `"all"` **with quotes**. It is a piece of text.

Choosing the tasks

```
function getVisibleTasks() {
  const visibleTasks = [];

  for (const task of tasks) {
    if (currentFilter === "all") {
      visibleTasks.push(task);
    } else if (currentFilter === "completed" && task.completed) {
      visibleTasks.push(task);
    } else if (currentFilter === "pending" && !task.completed) {
      visibleTasks.push(task);
    }
  }

  return visibleTasks;
}
```

Step by step:

1. Start with an empty list.

```
const visibleTasks = [];
```

2. Look at each task and decide.

- If the filter is `"all"` -> keep every task.

- If the filter is "completed" **and** the task is completed -> keep it.
- If the filter is "pending" **and** the task is **not** completed -> keep it.

`&&` means **and**. Both sides must be true.

`!task.completed` means **not** completed.

3. `push()` adds a task to the new list.

4. `return` gives the finished list back to whoever asked for it.

The most important idea here

We **never delete anything** from `tasks` .

We build a **new, smaller list**. The original `tasks` array always keeps all six tasks.

That is why you can click All, then Completed, then All again, and everything still works. If we deleted tasks to filter them, they would be gone forever.

23. The Active Filter Button

When you click a button, it turns blue. That is the `active` class from our CSS.

JavaScript does not set colours. It only adds and removes the class:

```
function setFilter(newFilter, clickedButton) {
  currentFilter = newFilter;

  filterAllButton.classList.remove("active");
  filterCompletedButton.classList.remove("active");
  filterPendingButton.classList.remove("active");

  clickedButton.classList.add("active");

  renderTasks();
}
```

`classList` is the list of CSS classes on an element.

- `classList.remove("active")` takes the class off.
- `classList.add("active")` puts it on.

Notice the order: we remove `active` from **all three** buttons first, then add it to the one that was clicked. That is how we make sure only one button is ever blue.

Here is the flow:

```
JavaScript adds "active"
    |
    v
CSS rule .filter-button.active is used
    |
    v
The button looks blue
```

24. Why `classList` and Not Inline Styles?

JavaScript could change the colour directly:

```
button.style.backgroundColor = "#3b6ef5";
button.style.color = "#ffffff";
button.style.borderColor = "#3b6ef5";
```

Instead we wrote:

```
button.classList.add("active");
```

Why we prefer the class:

- 1. CSS keeps the design.** All the colours stay in `style.css`, where design belongs. If you want a different blue, you change one CSS rule and you are done.
- 2. One class can hold many rules.** Our `.filter-button.active` rule changes three things at once. With inline styles we would need three lines of JavaScript, and three more to undo them.
- 3. JavaScript only decides the state.** JavaScript says "this button is the active one". CSS decides what "active" looks like. Each language does its own job.
- 4. Undoing is easy.** `classList.remove("active")` puts everything back in one line.

To be fair: changing `style` from JavaScript **does work**, and sometimes it is the right tool — for example when a value is calculated and cannot be written in CSS ahead of time. Here, our design was already described in CSS, so using the class is cleaner.

25. Separating Data from the HTML

This is the most important idea of Session 2.

Before (Session 1 and the homework):

index.html

|

v

The six task rows were already written there by hand

Now (Session 2):

tasks array (in JavaScript)

|

v

renderTasks()

|

v

Task rows appear on the page

In `index.html`, the list is now empty:

```
<ul class="task-list" id="taskList"></ul>
```

It looks strange the first time. But the HTML now describes the **shape** of the page, and JavaScript provides the **content**.

Why is this better?

Because the data lives in **one** place.

- Want to add a task? Add one object to the array. The list, the total, the completed count and the pending count all update by themselves.
- Want to fix a spelling mistake in a title? Change it in one place.

Before, you would have to edit the HTML row **and** remember to fix the numbers by hand.

Say it like this: **the data is the truth, and the page is a picture of the data**. When the data changes, we draw the picture again.

26. Why an Array of Objects Instead of Many Variables?

Compare these two ideas.

Many separate variables:

```
const task1Title = "Design the TaskFlow page";
const task1Completed = true;
const task2Title = "Write the HTML structure";
const task2Completed = true;
```

One array of objects:

```
const tasks = [
  { id: 1, title: "Design the TaskFlow page", completed: true },
  { id: 2, title: "Write the HTML structure", completed: true },
];
```

Why the second one is better:

- Each task is **one object**, so its information stays together.
- All tasks are in **one array**, so they stay together too.
- One loop can handle every task.
- Adding a task means adding one object. Nothing else changes.

With separate variables, adding a task means new variable names, and every piece of code that counts or draws tasks has to be updated by hand.

27. Why Functions Instead of One Long Script?

We could have written everything as one long list of instructions from top to bottom.

Instead we made four functions with clear names:

```
updateStats();
getVisibleTasks();
renderTasks();
setFilter();
```


Why this helps:

- **You can read the names and understand the program.** `updateStats` counts. `renderTasks` draws. You do not have to read the code to know that.
 - **You can find problems faster.** Wrong numbers? Look in `updateStats()`.
 - **You can reuse code.** `renderTasks()` is called from four places. We wrote it once.
 - **Each part has one job.** `renderTasks()` does not decide which tasks to show — it asks `getVisibleTasks()`. Small jobs are easier to get right.
-

28. Why Loops Instead of Repeated Code?

Repeated code:

```
handle task 1
handle task 2
handle task 3
handle task 4
handle task 5
handle task 6
```

A loop:

```
for (const task of tasks) {
  // handle this task
}
```

The number of tasks can change. The loop does not care.

Copy-pasted code has another problem: if you find a mistake, you have to fix it in every copy. With a loop, there is only one place to fix.

29. Why Did We Write It This Way?

Here is a summary of the choices we made today.

We used	Another valid option	Why we used ours
<code>querySelector()</code>	<code>getElementById()</code>	One consistent CSS-selector style for ids and classes

We used	Another valid option	Why we used ours
<code>addEventListener()</code>	<code>onclick=""</code> in HTML	Keeps HTML and JavaScript separate
<code>const</code> by default	<code>let</code> everywhere	Shows which values are never reassigned
An array of tasks	Many separate variables	Keeps the tasks together, lets one loop handle all
An object per task	Separate values per task	Keeps one task's information together
A loop	The same code repeated	Works for any number of tasks
Small functions	One long script	Each job has a clear name
<code>textContent</code> for numbers	<code>innerHTML</code>	Plain text does not need HTML
<code>innerHTML</code> for rows	<code>textContent</code>	Our string really does contain HTML tags
Template literals	Joining with <code>+</code>	Much easier to read when building HTML
<code>classList</code>	Setting <code>style</code> directly	Lets CSS keep control of the design
<code>for...of</code>	<code>forEach()</code>	Very direct and easy to read for beginners

Please remember: the options on the right are **not wrong**. They are real tools that real developers use. We picked ours to keep the project simple and consistent while you are learning.

30. How the Project Changed

Before Session 2

- The six task rows were written in the HTML.
- The numbers 6, 2 and 4 were typed in the HTML.
- The buttons looked nice but did nothing.
- Adding a task meant writing more HTML by hand.

After Session 2

- The tasks are stored in a JavaScript array.
- JavaScript counts them and writes the three numbers.
- JavaScript builds every task row.
- The filter buttons react to clicks.

- `currentFilter` remembers which filter is selected.
- The clicked button gets the `active` class and turns blue.
- Adding a task means adding one object to the array.

The page looks almost the same as yesterday. But yesterday we typed it. Today we calculated it.

31. How Session 2 TaskFlow Works

When the page opens:

```
Browser opens index.html
      |
      v
The browser loads css/style.css
      |
      v
The browser loads js/app.js
      |
      v
The tasks array is created
      |
      v
The elements are found with querySelector()
      |
      v
updateStats()  ->  the three numbers appear
      |
      v
renderTasks()  ->  the six task rows appear
```

When the user clicks "Completed":

```
User clicks the Completed button
      |
      v
The click event happens
      |
      v
setFilter("completed", filterCompletedButton)
      |
```

```
      v
currentFilter becomes "completed"
      |
      v
"active" moves to the Completed button
      |
      v
renderTasks()
      |
      v
getVisibleTasks() keeps only completed tasks
      |
      v
Only the completed tasks appear
```

Notice that the three numbers do **not** change when you filter. They always describe **all** the tasks. Filtering changes what is **shown**, not what **exists**.

32. Session 2 HTML and CSS Changes

We did not rewrite the design today. We only made small changes so JavaScript could work.

1. We added ids

```
<p class="stat-value" id="totalCount">0</p>
<p class="stat-value" id="completedCount">0</p>
<p class="stat-value" id="pendingCount">0</p>
```

```
<button class="filter-button active" id="filterAll">All</button>
<button class="filter-button" id="filterCompleted">Completed</button>
<button class="filter-button" id="filterPending">Pending</button>
```

```
<ul class="task-list" id="taskList"></ul>
```

Why? JavaScript needs a reliable way to find one exact element.

The class stays for the design. The id is the name JavaScript uses:

```
document.querySelector("#totalCount");
```

Remember the difference:

- A **class** can be on many elements. It is for styling.
- An **id** must be on only one element. It is for finding that one element.

2. We changed the numbers to 0

The cards start at `0` instead of `6`, `2` and `4`.

Why? Because those numbers were hand-written answers. JavaScript replaces them the moment the page opens. If you ever see `0` on the screen, it means the JavaScript did not run — that is a useful clue.

3. We emptied the task list

```
<ul class="task-list" id="taskList"></ul>
```

All six `<li>` blocks were removed from the HTML. JavaScript builds them now.

4. The CSS did not change

This is worth noticing. Our CSS from Session 1 and the homework still works perfectly, because `renderTasks()` builds rows with the **same class names** we already had: `task-item`, `task-title`, `task-status`, `completed`, `pending`.

Good class names paid off.

33. Complete JavaScript Walkthrough

Here is the whole file, in the order it appears.

Part 1 - The task data

```
/* TaskFlow
   Our tasks live inside this file for now. */

// Every task is an object with three pieces of information.
const tasks = [
```

```
{
  id: 1,
  title: "Design the TaskFlow page",
  completed: true,
},
{
  id: 2,
  title: "Write the HTML structure",
  completed: true,
},
{
  id: 3,
  title: "Style the statistic cards",
  completed: false,
},
{
  id: 4,
  title: "Build the task list",
  completed: false,
},
{
  id: 5,
  title: "Practice JavaScript",
  completed: false,
},
{
  id: 6,
  title: "Learn how an API works",
  completed: false,
},
];
```

What it does: creates our list of six tasks.

Why it exists: this is our data. Everything else on the page is built from it.

What you see: nothing yet. Data alone does not change the page.

Part 2 - The current filter

```
let currentFilter = "all";
```

What it does: remembers which filter is selected.

Why `let` : it changes when the user clicks a button.

What you see: nothing. But it decides what `renderTasks()` will draw.

Part 3 - Finding the elements

```
// The parts of the page that JavaScript needs to change.
const taskList = document.querySelector("#taskList");

const totalCount = document.querySelector("#totalCount");
const completedCount = document.querySelector("#completedCount");
const pendingCount = document.querySelector("#pendingCount");

const filterAllButton = document.querySelector("#filterAll");
const filterCompletedButton = document.querySelector("#filterCompleted");
const filterPendingButton = document.querySelector("#filterPending");
```

What it does: finds the seven elements JavaScript needs to change or listen to.

Why it exists: we look them up once, at the top, and reuse the names everywhere below.

What you see: nothing yet.

Part 4 - Counting the tasks

```
// Count the tasks and write the numbers into the three cards.
function updateStats() {
  let completed = 0;
  let pending = 0;

  for (const task of tasks) {
    if (task.completed) {
      completed++;
    } else {
      pending++;
    }
  }

  totalCount.textContent = tasks.length;
```

```
completedCount.textContent = completed;
pendingCount.textContent = pending;
}
```

What it does: counts completed and pending tasks, then writes all three numbers.

Why it exists: so the numbers are calculated instead of typed.

What you see: when this runs, the cards change from 0, 0, 0 to 6, 2, 4.

Part 5 - Choosing the visible tasks

```
function getVisibleTasks() {
  const visibleTasks = [];

  for (const task of tasks) {
    if (currentFilter === "all") {
      visibleTasks.push(task);
    } else if (currentFilter === "completed" && task.completed) {
      visibleTasks.push(task);
    } else if (currentFilter === "pending" && !task.completed) {
      visibleTasks.push(task);
    }
  }

  return visibleTasks;
}
```

What it does: builds a new, smaller list based on `currentFilter`.

Why it exists: so the drawing code does not have to think about filters. One job each.

What you see: nothing directly. It gives its answer to `renderTasks()`.

Part 6 - Drawing the tasks

```
// Build the HTML for every task and put it on the page.
function renderTasks() {
  const visibleTasks = getVisibleTasks();

  let html = "";
```



```

for (const task of visibleTasks) {
  let statusClass = "pending";
  let statusText = "Pending";

  if (task.completed) {
    statusClass = "completed";
    statusText = "Completed";
  }

  html += `
    <li class="task-item">
      <span class="task-title">${task.title}</span>
      <span class="task-status ${statusClass}">${statusText}</span>
    </li>
  `;
}

taskList.innerHTML = html;
}

```

What it does: builds one `<li>` for each visible task, then puts them all on the page at once.

Why it exists: so we write the row HTML once instead of six times.

What you see: the task rows appear, with the right colours and words.

Part 7 - Changing the filter

```

function setFilter(newFilter, clickedButton) {
  currentFilter = newFilter;

  filterAllButton.classList.remove("active");
  filterCompletedButton.classList.remove("active");
  filterPendingButton.classList.remove("active");

  clickedButton.classList.add("active");

  renderTasks();
}

```

What it does: three things — remember the filter, move the blue colour, draw again.

Why it exists: all three buttons need exactly these steps, so we wrote them once and pass in what is different.

What you see: the clicked button turns blue and the list changes.

Part 8 - Listening for clicks

```
filterAllButton.addEventListener("click", function () {
  setFilter("all", filterAllButton);
});

filterCompletedButton.addEventListener("click", function () {
  setFilter("completed", filterCompletedButton);
});

filterPendingButton.addEventListener("click", function () {
  setFilter("pending", filterPendingButton);
});
```

What it does: connects each button to `setFilter` with its own values.

Why it exists: without listeners, the buttons would still do nothing.

What you see: nothing until you click. Then the page reacts.

Part 9 - Starting the page

```
// Show the page for the first time.
updateStats();
renderTasks();
```

What it does: runs our two main functions once, when the page opens.

Why it exists: functions do nothing until they are called. Without these two lines the page would stay empty forever.

What you see: the finished page — three numbers and six task rows.

34. Common Beginner Mistakes

These happen to everyone. Here is how to spot and fix them.

1. Forgetting the

```
document.querySelector("taskList");    // wrong
document.querySelector("#taskList");   // correct
```

Without `#`, JavaScript looks for a `<taskList>` tag, which does not exist. It returns `null`, and the next line fails with **"Cannot read properties of null"**.

2. A spelling mistake in the id

```
document.querySelector("#taskslist"); // wrong
document.querySelector("#taskList");  // correct
```

The id in the HTML and in the JavaScript must match **exactly**, including capital letters.

3. A spelling mistake in a property

```
task.complete    // wrong – gives undefined
task.completed   // correct
```

This one does not crash. It just quietly behaves as if every task is pending. Wrong output with no error message usually means a wrong name.

4. Forgetting the quotes

```
currentFilter = completed;    // wrong – JavaScript looks for a variable
currentFilter = "completed";  // correct – this is text
```

5. Mixing up `true` and `"true"`

```
completed: "true"    // this is text, not a boolean
completed: true      // correct
```

`if (task.completed)` is true for **any** non-empty text, so `"false"` would also count as completed. Confusing to find.

6. Using `=` instead of `===`

```
if (currentFilter = "all")    // wrong - this SETS the value
if (currentFilter === "all") // correct - this ASKS
```

- `=` means "put this value in here".
- `===` means "are these the same?"

7. Forgetting to call the function

```
function updateStats() {
  ...
}
// and nothing else
```

The page stays at 0, 0, 0 and no error appears. Writing a function is not the same as running it.

8. Using a variable before finding the element

If you write `taskList.innerHTML = html;` **above** the line that creates `taskList`, it fails. Find your elements at the top of the file.

9. Using `const` for something that changes

```
const currentFilter = "all";
currentFilter = "completed"; // Error: Assignment to constant variable
```

If a value needs to change, use `let`.

10. A missing comma in the array

```
const tasks = [
  { id: 1, title: "A", completed: true } // missing comma
  { id: 2, title: "B", completed: false },
];
```

The whole file stops working. Every item in an array needs a comma after it, except the last one (a comma there is allowed too).

11. A missing bracket

Every `{` needs a `}`. Every `(` needs a `)`. In VS Code, click on a bracket and it will show you its partner.

12. Using normal quotes instead of back-ticks

```
html += "<span>${task.title}</span>"; // shows ${task.title} as text
html += `${task.title}</span>`; // correct
```

13. Forgetting to save or refresh

If nothing changes at all, check for the white dot next to the file name in VS Code, then refresh the browser.

14. Ignoring the Console

The Console tells you the problem **and** the line number. Read it before guessing.

35. A Simple Way to Debug JavaScript

When something does not work, follow these steps in order.

1. **Save the file.** An unsaved change does nothing.
2. **Refresh the browser.** You may be looking at the old version.
3. **Open DevTools.** Press `F12`, or right-click the page and choose Inspect.
4. **Open the Console tab.** This is where JavaScript reports problems.
5. **Look at the first red error.** Only the first one matters. The others are often caused by it.
6. **Check the file name and line number.** The error tells you exactly where to look. Click it to jump there.
7. **Check spelling.** Ids, property names, variable names. They must match exactly.
8. **Check brackets and parentheses.** Every `{` needs a `}`. Every `(` needs a `)`.
9. **Use `console.log()` to check a value.**

`console.log()` prints a value in the Console. It helps you see what JavaScript is really doing.

```
console.log(currentFilter);
console.log(tasks.length);
console.log(getVisibleTasks());
```

For example, put this at the top of `getVisibleTasks()` and click a button:

```
console.log(currentFilter);
```

Is the value what you expected? If not, the problem is above that line.

10. Test again.

Remember to remove your `console.log()` lines when you are finished. They are a tool, not part of the final program.

Common error messages

Message	What it usually means
Cannot read properties of null	<code>querySelector()</code> found nothing. Check the id and the <code>#</code> .
x is not defined	A spelling mistake, or the variable does not exist yet.
Assignment to constant variable	You used <code>const</code> for something that changes. Use <code>let</code> .
Unexpected token	A missing comma, bracket or quote.
No error, but wrong numbers	A comparison is wrong. Check <code>=</code> versus <code>===</code> , and property names.
No error, nothing happens	A function was never called.

36. Quick Review Questions

Try to answer without looking. The answers are on the next pages.

1. What does JavaScript add to our page?
2. What is a variable?
3. What is the difference between `const` and `let`?
4. What is an array?
5. Why do we store all the tasks in one array?

6. What is an object?
 7. What does `completed: true` mean?
 8. What is a boolean?
 9. Why do we use a loop instead of writing the same code six times?
 10. What does `querySelector()` do?
 11. Why does `"#taskList"` start with `#` ?
 12. Why did we choose `querySelector()` instead of `getElementById()` ?
 13. What is a function, and what must you do to make it run?
 14. What does `renderTasks()` do?
 15. What is a parameter?
 16. What does `addEventListener()` do?
 17. Why did we choose `addEventListener()` instead of `onclick` ?
 18. Why do we need `currentFilter` ?
 19. What happens, step by step, when the Completed button is clicked?
 20. What is the difference between `textContent` and `innerHTML` ?
 21. What does `classList.add("active")` do?
 22. Why do we build a new list in `getVisibleTasks()` instead of deleting tasks?
-

37. Answers

1. Behaviour and logic. JavaScript stores the data, calculates the numbers, builds the task rows, and reacts when the user clicks.
2. A name that stores a value.
3. `const` cannot be given a new value. `let` can. Start with `const` , and use `let` only when the value must change.
4. A list of values, written with `[ ]` and commas between the items.
5. Because the tasks belong together. One array lets one loop handle every task, and adding a task does not mean changing the rest of the code.
6. A group of related values, written with `{ }` . Each value has a name, called a property.
7. That this task is finished.
8. A value that can only be `true` or `false` .
9. Because the loop works for any number of tasks, and there is only one place to fix if something is wrong.

10. It finds one element on the page, using a CSS selector.
 11. Because `#` means "id" in a CSS selector, exactly like in CSS.
 12. It uses the same selector language we already know from CSS, and one function works for ids and classes. `getElementById()` is also valid — we chose `querySelector()` for consistency.
 13. A group of code that does one job. You must **call** it, for example `updateStats();`. Writing it is not enough.
 14. It asks which tasks should be visible, builds the HTML for each one, and puts the result inside the task list.
 15. A value that a function receives. In `setFilter(newFilter, clickedButton)`, both names are parameters.
 16. It tells JavaScript: when this event happens on this element, run this code.
 17. To keep HTML and JavaScript separate. HTML describes the page, JavaScript describes the behaviour. `onclick` also works, but mixes the two together.
 18. It remembers which filter is selected, so `getVisibleTasks()` knows which tasks to keep.
 19. The click event happens -> `setFilter("completed", ...)` runs -> `currentFilter` becomes "completed" -> `active` moves to that button -> `renderTasks()` runs -> only completed tasks appear.
 20. `textContent` sets plain text. `innerHTML` sets text that the browser reads as HTML. We use `textContent` for the numbers and `innerHTML` for the task rows.
 21. It adds the CSS class `active` to that element, so the CSS rule for `.active` starts to apply and the button turns blue.
 22. Because the original list must stay complete. If we deleted tasks, they would be gone forever and clicking All again would show nothing.
-

38. Glossary

JavaScript The language that adds behaviour to a page. It can store data, make decisions and change what you see.

variable A name that stores a value.

const Creates a variable that cannot be given a new value later.

let Creates a variable whose value can change later.

string A piece of text, written inside quotes, like `"all"`.

number A value like `6` or `0` . No quotes.

boolean A value that is only `true` or `false` .

array A list of values, written with `[ ]` .

object A group of related values, written with `{ }` .

property One named value inside an object, like `title` or `completed` .

`push()` Adds an item to the end of an array.

`length` How many items an array has.

loop Code that runs once for every item in a list.

condition A check, written with `if` . It decides whether some code runs.

function A group of code that does one job and has a name.

call To run a function, by writing its name with `()` after it.

parameter A value that a function receives when it is called.

return What a function gives back to the code that called it.

DOM The page as JavaScript sees it: a tree of elements it can read and change.

`document` The current webpage in JavaScript.

selector The text we use to find an element, like `"#taskList"` or `".filter-button"` .

`querySelector()` Finds one element on the page using a selector.

event Something that happens on the page, like a click.

event listener Code that waits for an event and then runs.

`addEventListener()` Connects an element, an event, and the code to run.

render To build and show something on the page. `renderTasks()` renders the task rows.

filter To show only some of the items, based on a rule.

`classList` The list of CSS classes on an element. JavaScript can add and remove classes with it.

template literal A string written with back-ticks that can hold values inside `${ }` and span many lines.

`textContent` Changes the plain text inside an element.

`innerHTML` Changes the contents of an element, and the browser reads it as HTML.

`console.log()` Prints a value in the browser Console, so you can see what your code is doing.

You Are Done

You now understand every line of the Session 2 TaskFlow project.

You can:

- store data in arrays and objects,
- find elements on the page with `querySelector()` ,
- write functions with clear jobs,
- use loops and conditions,
- build HTML with template literals,
- react to clicks with `addEventListener()` ,
- change classes with `classList` ,
- and explain **why** we made each of these choices.

The biggest idea of today: **the data is the truth, and the page is a picture of the data.**

In the next session, our tasks will stop living inside our own file. We will ask another computer on the internet to give them to us.

See you next session.